Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

Übung 08: Arithmetik

M.Sc. Jasper Homann, M.Sc. Eike Trumann

Prof. Dr.-Ing. Guillermo Payá Vayá

12. Dezember 2025

1 Arithmetik in VHDL

Kurzfragen

Welchen Wertebereich kann folgendes signed Signal darstellen (*data*)?

```
signal data : signed(8 - 1 downto 0);
```

Welchen Wertebereich kann folgendes unsigned Signal darstellen (*data*)?

```
signal data : unsigned(5 - 1 downto 0);
```

Wie breit ist das Ergebnissignal der Addition der folgenden beiden unsigned Zahlen ($a + b$)?

```
signal a, b : unsigned(8 - 1 downto 0);
```

Wie breit ist das Ergebnissignal der Multiplikation der folgenden beiden signed Zahlen ($a \cdot b$)?

```
signal a, b : signed(32 - 1 downto 0);
```

Das signed Fixed Point Signal a (*4 Integer-Stellen, 4 Nachkomma-Stellen*) wird auf die Zahl b (*5 Integer-Stellen, 5 Nachkomma-Stellen*) addiert ($a + b$). Wie viele Stellen muss das Ergebnissignal haben?

$$a_{\{4.4\}} + b_{\{5.5\}} = c \ ___ \ . \ ___$$

Integer Berechnungen

Stellen Sie die Zahl $a = -113_{10}$ als 2-er Komplementzahl mit 8 Stellen dar.
Stellen Sie außerdem die Zahl $b = 17_{10}$ als 2-er Komplementzahl dar.

$$a = _\ _\ _\ _\ _\ _\ _\ _$$

$$b = _\ _\ _\ _\ _\ _\ _\ _$$

Addieren Sie $a + b$ und Stellen Sie das Ergebnis c mit 8 Stellen dar.

$$c = _\ _\ _\ _\ _\ _\ _\ _$$

Addieren Sie $a + a$ und Stellen Sie das Ergebnis d mit 8 Stellen dar.

$$d = _\ _\ _\ _\ _\ _\ _\ _$$

Ist das dargestellte Ergebnis korrekt?

Fixed Point Berechnungen

Stellen Sie die Zahl $a = -105.75_{10}$ als 2-er Komplementzahl mit 8 Integer-Stellen und 8 Nachkommastellen dar.
Stellen Sie außerdem die Zahl $b = 19.1875_{10}$ als 2-er Komplementzahl mit 8 Integer-Stellen und 8 Nachkommastellen dar.

$$a = _\ _\ _\ _\ _\ _\ _\ _ \cdot _\ _\ _\ _\ _\ _\ _\ _$$

$$b = _\ _\ _\ _\ _\ _\ _\ _ \cdot _\ _\ _\ _\ _\ _\ _\ _$$

Addieren Sie $a + b$ und Stellen Sie das Ergebnis c mit 8 Integer-Stellen und 8 Nachkommastellen dar.

$$c = _\ _\ _\ _\ _\ _\ _\ _ \cdot _\ _\ _\ _\ _\ _\ _\ _$$

Ist das dargestellte Ergebnis korrekt?

2 ADDSUB Optimierung

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity ADDSUB is
        port( A_i , B_i  : in  std_ulogic_vector(4 downto 0);
              OP_i, C_i  : in  std_ulogic;
              SUM_o      : out std_ulogic_vector(4 downto 0));
end ADD_SUB;

architecture RTL of ADD_SUB is
begin
        ADDSUB : process(OP_i, C_i, A_i, B_i)
                variable ZW, V1, V2, V3: signed(4 downto 0);
        begin
                V1 := signed(A_i(4 downto 0));
                V2 := signed(B_i(4 downto 0));
                V3 := "0000" & signed(C_i);
                if OP_i = '1' then
                        ZW := V1 + V2 + V3; -- ADDITION
                else
                        ZW := V1 - V2 - V3; -- SUBTRAKTION
                end if;
                SUM_o <= std_ulogic_vector(ZW));
        end process ADD_SUB;
end architecture RTL;
```

Wie sieht die Hardwareschaltung zu dieser in VHDL implementierten Architektur aus? Wie kann diese Architektur optimiert werden?

3 Fixed Point Format in VHDL

Stellen Sie den Wert 5.75 in a, b und c dar, sowie den Wert -1.5 in d.

```
use IEEE.fixed_pkg.all;
use IEEE.fixed_float_types.all;
...
        signal a : ufixed(2 downto -2);
        signal b : sfixed(3 downto -4);
        signal c : sfixed(4 downto -3);
        signal d : sfixed(2 downto -3);
begin

        a <=

        b <=

        c <=

        d <=

end architecture;
```

Weisen Sie c das Ergebnis der Addition zu ($c = a + b$, konstant). Welche Werte stellen a, b und c dar (dezimal)?

```
        signal a : ufixed(3 downto -4);
        signal b : ufixed(3 downto -4);
        signal c : ufixed(3 downto -4);
begin
        a <= "0011" & "1100";
        b <= "0001" & "1000";

        c <=

end architecture;
```

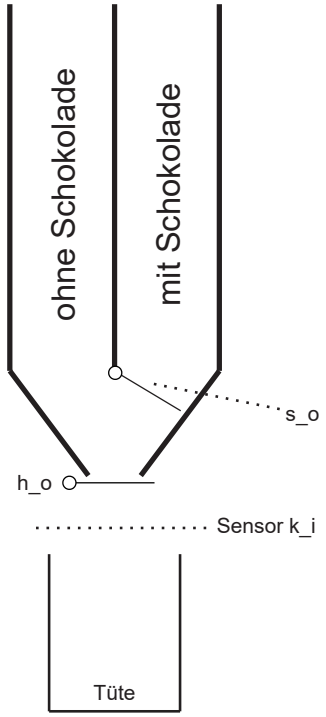
Welche Ergebniswerte nehmen c, d und e an?

```
        signal a : ufixed(3 downto -4);
        signal b : ufixed(3 downto -8);
        signal c : ufixed(3 downto -4);
        signal d : ufixed(3 downto -4);
        signal e : ufixed(3 downto -4);
begin
        a <= "0001" & "1000";
        b <= "1111" & "10101111";

        c <= resize(a + b, c, fixed_wrap, fixed_truncate);
        d <= resize(a + b, d, fixed_wrap, fixed_round);
        e <= resize(a + b, d, fixed_saturate, fixed_round);

end architecture;
```

Wie groß ist das Ergebnissignal c im folgenden Codeabschnitt?

```
c <= resize(a(3 downto -4) * b(2 downto -4));
```

4 FSM und Zähler



Es soll die Steuerung einer Verpackungsmaschine entworfen werden. Die Maschine füllt kleine gemischte Tüten mit Keksen. In jeder Tüte sollen erst 13 Kekse mit Schokolade und dann 7 Kekse ohne Schokolade verpackt werden. Die Kekse fallen aus einem Vorratsbehälter einzeln direkt in die Tüte. Über das Signal **s_o** der Steuerung kann zwischen den Kammern der Kekse umgeschaltet werden ($s_o = 1$ mit Schokolade, $s_o = 0$ ohne Schokolade). Ein Sensor zeigt durch das Signal **k_i**= 1 an, dass im aktuellen Takt ein Keks in die Tüte gefallen ist. Sobald die gewünschte Kekszahl einer Sorte in der Tüte ist, soll durch den Ausgang **h_o**= 1 (hold) das Abfüllen weiterer Kekse für einen Takt verhindert werden und auf die andere Kekssorte umgeschaltet werden. Nach dem letzten Keks zeigt die Steuerung durch **t_o**= 1 an, dass die Tüte voll ist. t_o soll dabei gleichzeitig zu h_o gesetzt sein. Die Maschine verschließt dann die Tüte und lädt eine neue.

Tipp: In Takten, in denen h_o gesetzt ist, fällt kein Keks in die Tüte.

Entwerfen Sie das Diagramm einer FSM, die die gewünschte Steuerung realisiert. Nutzen Sie dabei einen einzelnen Counter (Signal *counter_ff* + *counter_next*), um den Kekszähler zu implementieren. Implementieren Sie die FSM in VHDL.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;
entity keks_control is
        port( clk_i, areset_ni : in  std_ulogic;
              k_i              : in  std_ulogic;
              h_o, s_o, t_o    : out std_ulogic);
end keks_control;
```

```
architecture RTL of keks_control is

begin

end architecture RTL;
```